

अग्र-Sandhani (Agra-Sandhani)

Enterprise Administrator Operations & Security Manual

RESTRICTED DOCUMENTATION

This manual is intended solely for system administrators, security officers, and IT infrastructure teams managing the **Agra-Sandhani** deployment. It contains details regarding security auditing, database purges, IP tracking, and data retention configurations.

1. Administrative Privileges Overview

Administrators manage system health, user lifecycle, and data policies. Admin actions are defined in the `AdminDashboardPage.jsx` interface and backed by the admin controllers at `admin-users.js` and `admin-analytics.js`.

2. User Governance & Security Auditing

Admins oversee user lifecycle management and monitor for anomalies:

A. Creating & Provisioning Accounts

- **Action:** Add new users directly via the admin console.
- **Security Protocol:** New accounts require a username and a password of at least 6 characters. Passwords are hashed using `bcrypt` with a work factor of 12 rounds before database insertion.

B. Updating & Demoting Accounts

- **Password Reset:** Force-reset passwords for users locked out or compromised.
- **Role Changes:** Promote standard users to `admin` or demote administrators to standard `1 user` roles.
 - *Safety Constraint:* The system blocks admins from demoting themselves to prevent administrator lockouts.

C. Active Session Monitoring

From the **Active Sessions** tab, administrators monitor real-time database logins:

- `current_session_id` : Displays active authentication tokens.
- `last_activity` : Real-time tracking of the user's latest interaction with the system.
- `last_login_ip` / `registration_ip` : Captures host IP addresses.

SECURITY NOTE

In secure intranet environments (e.g., DRDO's Drona network), cross-referencing registration and login IPs is crucial for detecting unauthorized device bridging or IP spoofing.

3. Form Access Permissions & Global Delegations

Admins manage the security permissions of shared dynamic forms:

- **Pending Access Requests:** The approvals queue lists standard users requesting access to forms owned by other departments. Admins can globally override ownership boundaries to grant or deny access.
- **Global Proxy Delegations:** From the delegations panel, admins can establish access links between standard users (e.g., granting `User B` access to `User A`'s forms during leaves).
 - *Configuration:* Admin selects the Grantor (form owner), the Grantee (recipient), and selects a delegation timeframe (hours, days, or a specific calendar date).
 - *Revocation:* Admins can terminate delegations immediately using the `Revoke` button.

4. Database Deletion Scenarios & Retention Policies

Agra-Sandhani separates deletions into user-controlled **Soft Deletes**, administrator-controlled **Hard Purges**, and automated **Retention Purges**.

User Action: Click Delete → `Soft Delete` (Trash Bin: `deleted_at` timestamp set)

—— **Admin Action:** Restore → Restored to Active View

—— **Admin Action:** Purge Item / Empty Trash → Hard Delete (`DELETE FROM DATABASE`)

—— **Automated System:** 30-Day Cron Expiration → Hard Delete (`DELETE FROM DATABASE`)

Deletion Case Matrix

Deletion Type	Database Action	User Visibility	Admin Visibility	Audit Trail Status
Standard User Form/Record Delete	Sets <code>deleted_at</code> to timestamp	Hidden immediately	Visible in Trash Bin tab	Logged (Retains original record)
Admin Purge Item (Row/Form)	<code>DELETE FROM</code> (Hard Delete)	Hidden permanently	Removed permanently	Logged as <code>purge_item</code>
Admin Empty Trash Bin	Truncates soft-deleted items	Hidden permanently	Removed permanently	Logged as <code>empty_trash</code>
Admin Delete User	<code>DELETE FROM users WHERE id = X</code>	Blocked from login	Removed from system	Logged as <code>delete_user</code> (All owned forms/submissions hard-deleted)
Automated 24h Cron Purge	Hard-deletes items older than 30 days	Hidden permanently	Removed permanently	Logged under server runtime logs

A. The Soft-Deletion State

When a user deletes a submission or form, the system runs an update query setting the `deleted_at` field to the current timestamp.

- **Form Recovery:** The form configuration, field versions, and submissions are preserved in the database.
- **Submission Recovery:** The JSONB data payload (`data_json`) is preserved. The record is removed from analytical query interfaces, but the audit history is retained.

B. Trash Bin Restorations

In the admin dashboard's **Trash Bin** tab, soft-deleted forms and submissions can be restored:

- **Restore Button:** Reverses the deletion by updating the database record:

```
UPDATE submissions SET deleted_at = NULL WHERE id = $1;  
UPDATE forms SET deleted_at = NULL WHERE id = $1;
```

- Restoring a record automatically logs a `restore_submission` or `restore_form` event, linking it to the administrator who authorized the recovery.

C. Hard-Purging (Permanent Deletions)

- **Purging a Submission:** Permanently deletes the submission row from the database.
- **Purging a Form:** Permanently deletes the form row. Because the database is configured with cascading foreign keys (`ON DELETE CASCADE`), purging a form automatically deletes:
 1. All version histories associated with the form.
 2. All fields mapped to those versions.
 3. All submissions recorded against those fields.
- **Empty Trash Button:** Cleans all soft-deleted records. It executes:

```
DELETE FROM submissions WHERE deleted_at IS NOT NULL;  
DELETE FROM forms WHERE deleted_at IS NOT NULL;
```

D. Data Retention Policy & Automated Purge Cron Job

To optimize database performance and comply with security guidelines, the Express server runs an automated trash purge check.

- **Frequency:** Checks once every **24 hours** using a JavaScript interval loop:

```
setInterval(async () => {  
    // Runs automatically every 24 hours  
}, 24 * 60 * 60 * 1000);
```

- **SQL Execution:** The background job runs the following queries:

```
-- Purge submissions soft-deleted for over 30 days
DELETE FROM submissions WHERE deleted_at < NOW() - INTERVAL '30 days';

-- Purge forms soft-deleted for over 30 days
DELETE FROM forms WHERE deleted_at < NOW() - INTERVAL '30 days';
```

- **System Logs:** If any records are purged during the cron run, the details are logged:

```
[CRON] Purged X submissions and Y forms older than 30 days
```

- **Security Lockout:** Once deleted by the cron job, the data is removed from the database and cannot be recovered via the web interface.

PHYSICAL FILE RETENTION WARNING

When submissions containing document attachments (`file_upload` fields) are purged (either manually or by the 30-day cron job), the system deletes the database record link, but the **physical files remain stored on disk** inside the `/uploads/` directory.

This is a deliberate security design in air-gapped systems to preserve files for filesystem-level audits. To clean up disk storage:

1. Log in to the server hosting the application.
2. Open the `/server/uploads/` directory.
3. Verify the files against the database entries and manually delete folders starting with `batch_` or orphaned files.

5. System Health & Performance Monitoring

Admins use the **System Health** tab to identify bottlenecking queries, missing indexes, and database pool limits:

- **Missing Index Analytics:** Monitors table scan statistics. If index scan ratios drop, it indicates the database is performing sequential reads instead of using indexes.
 - *Correction:* Database indexes (GIN and Trigram) are defined in:
 - `add-gin-indexes.js`
 - `add-trigram-indexes.js`
 - `add-scaling-indexes.js`
- **Database Pool Utilization:** Monitors active and idle connection counts in the PostgreSQL connection pool to prevent connection exhaustion under high concurrency.
- **Table Size Monitor:** Tracks database storage consumption. Large forms with many submissions will show increased JSONB storage size.

6. PM2 Production Process Management

In production, the backend is orchestrated by **PM2** to ensure automated restarts, logging, and cluster monitoring.

PM2 Process File

Configured in `ecosystem.config.js`:

```
module.exports = {
  apps : [{
    name: 'agra-sandhani-server',
    script: 'server/index.js',
    instances: 'max',
    exec_mode: 'cluster',
    env: {
      NODE_ENV: 'production',
    }
  }]
}
```

Essential PM2 Commands for Administrators

Run these commands from the terminal in the application directory:

- **View Process Status:** Check if the backend is running and monitor CPU/RAM usage:

```
pm2 status
```

- **Monitor Real-time Log Stream:** View API traffic and cron outputs:

```
pm2 logs agra-sandhani-server
```

- **Restart the App Pool:** Trigger a rolling restart without downtime (useful after applying migrations or config updates):

```
pm2 reload agra-sandhani-server
```

- **Stop the Server:** Terminate server operations immediately:

```
pm2 stop agra-sandhani-server
```

- **Startup Configuration:** Save the process list so the backend restarts automatically if the host machine reboots:

```
pm2 save
```